

`uvm_object_utils(lpi_basic_seq)` is a **UVM factory registration macro** used in **SystemVerilog UVM**.

It tells UVM:

- “This class is a UVM object”
- “Register this class with the UVM factory”
- “Allow this object to be created dynamically using the factory methods”

```
class lpi_basic_seq extends uvm_sequence;

    `uvm_object_utils(lpi_basic_seq)

    function new(string name = "lpi_basic_seq");
        super.new(name);
    endfunction

endclass
```



This macro automatically generates code internally for:

1. Factory registration

So UVM can recognize the class.

2. create() support

Allows object creation like:

Why factory is important?

Factory enables:

- Object overriding
- Reusability
- Test customization
- Polymorphism

```
set_type_override_by_type(base_seq::get_type(),
                           error_seq::get_type());
```

Now wherever `base_seq` is created using factory, `error_seq` gets created automatically.

Difference between object and component macros:

For sequence/items/transactions (objects):

```
`uvm_object_utils(my_item)
```

Used with:

- sequence
- sequence_item
- transaction
- config object

For components:

``uvm_component_utils(my_driver)`..... for components with hierarchy + phases.

Used with:

- driver
- monitor
- agent
- env
- test

Because components have hierarchy and phases.

Why phases are needed?

A UVM testbench has many components:

- test
- env
- agent
- driver
- monitor
- scoreboard

If everyone starts working randomly, chaos happens. So UVM divides simulation into ordered steps called phases.

Common UVM phases

1. Build Phase: Used to create components.

```
function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    drv = my_driver::type_id::create("drv", this);
endfunction
```

2. Connect Phase: Connect TLM ports/exports.

```
function void connect_phase(uvm_phase phase);
    drv.seq_item_port.connect(seqr.seq_item_export);
endfunction
```

3. Run Phase: Actual simulation activity happens here.

- driving signals
- generating packets
- monitoring DUT

```
task run_phase(uvm_phase phase);

    forever begin
        seq_item_port.get_next_item(req);
        // drive DUT
        seq_item_port.item_done();
    end

endtask
```

4. Check Phase: Final checking after simulation.

5. Report Phase: Print pass/fail summaries.

Why only components have phases?

Because components are permanent hierarchical blocks of the testbench. These need synchronized execution. But **Objects** like:

- sequence_item
- transaction
- sequence

are temporary data objects. They are created/used/deleted dynamically. Think of UVM components like workers in a factory. Everyone follows same schedule. **Phases are:**

1. Build factory
2. Connect machines
3. Start production
4. Inspect products
5. Generate report

